

OPERATING SYSTEM COMPLETE NOTES

Easy Language • Real-Life Examples • Important Topics Highlighted

★ *Red star = Important Topic (read carefully, exam / interview favorite)*

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to Operating System

- 2. Types of Operating Systems

★ 3. Process

- 4. Process States and Process Control Block (PCB)

★ 5. Process Scheduling (FCFS, SJF, Round Robin)

★ 6. Threads

★ 7. CPU Scheduling Algorithms

- 8. Inter-Process Communication (IPC)

★ 9. Deadlocks

★ 10. Memory Management

★ 11. Paging and Segmentation

★ 12. Virtual Memory

★ 13. File Systems

- 14. Disk Scheduling

★ 15. Semaphores and Synchronization

- 16. System Calls

★ 1. Introduction to Operating System [IMPORTANT TOPIC]

Explanation:

An Operating System (OS) is special software that manages a computer's hardware and software resources, and acts as a bridge between the user and the computer. Without an OS, running any program or even seeing a screen display would not be possible.

Key Points:

- Manages hardware -> CPU, memory, storage, devices.
- Runs and manages programs (processes).
- Provides a user interface (like Windows, or a Linux terminal).
- Examples: Windows, macOS, Linux, Android.

Example:

When you open a web browser, the OS allocates memory for it, lets it use the CPU, and manages how it displays on your screen — all without you noticing.

Real-Life Example:

Every smartphone runs an OS too — Android and iOS are both operating systems specially built for mobile devices.

2. Types of Operating Systems

Explanation:

Different types of operating systems are designed for different needs, from single tasks to managing huge numbers of users at once.

Key Points:

- Batch OS -> processes jobs in batches, with no direct user interaction.
- Time-Sharing OS -> allows multiple users to use the computer at the same time by quickly switching between them.
- Real-Time OS -> responds to inputs within a strict, guaranteed time limit (used in medical devices, rockets).
- Distributed OS -> manages a group of separate computers as if they were one single system.

Example:

A hospital's life-support machine uses a Real-Time OS, because even a tiny delay in response could be dangerous.

Real-Life Example:

Modern desktop and mobile operating systems (Windows, Android) are mainly Time-Sharing systems, allowing you to run several apps that all seem to work at once.

★ 3. Process [IMPORTANT TOPIC]

Explanation:

A Process is a PROGRAM that is currently running. While a program is just a file sitting on the disk, a process is that program actively loaded into memory and being executed by the CPU.

Example:

Opening the same web browser twice creates TWO separate processes, each with its own memory space, even though both come from the same program file.

Real-Life Example:

Opening Task Manager (Windows) or Activity Monitor (Mac) shows you a live list of every process currently running on your computer.

4. Process States and Process Control Block (PCB)

Explanation:

A process moves through different STATES during its life — New, Ready, Running, Waiting, and Terminated. The OS tracks every detail about a process using a data structure called the Process Control Block (PCB).

Key Points:

- New -> the process is being created.
- Ready -> waiting for the CPU to become available.
- Running -> currently being executed by the CPU.
- Waiting -> paused, waiting for something like a file or user input.
- Terminated -> finished execution.

Example:

When you click 'Print', that process moves to the 'Waiting' state until the printer is ready, then returns to 'Ready' once it can continue.

Real-Life Example:

The PCB is like an ID card the OS keeps for every single running process, tracking its state, memory, and priority.

★ 5. Process Scheduling (FCFS, SJF, Round Robin) *[IMPORTANT TOPIC]*

Explanation:

Since a computer usually has far more processes than CPU cores, the OS must decide WHICH process runs next — this decision-making is called Scheduling.

Key Points:

- FCFS (First Come First Serve) -> processes run in the exact order they arrive.
- SJF (Shortest Job First) -> the process needing the LEAST time runs first.
- Round Robin -> each process gets a small, fixed 'time slice' before moving to the next, cycling repeatedly.

Example:

Round Robin scheduling is why you can type in a document AND listen to music AND browse the internet all seemingly 'at once' — the OS is rapidly switching between them.

Real-Life Example:

Most modern desktop operating systems use a more advanced version of Round Robin scheduling to keep the system feeling responsive to every open app.

★ 6. Threads [IMPORTANT TOPIC]

Explanation:

A Thread is the smallest unit of execution WITHIN a process. A single process can have MULTIPLE threads running at once, all sharing the same memory space — this is called 'multithreading'.

Example:

In a video game, one thread might handle graphics rendering, while another handles sound, and another listens for keyboard input — all running together inside the same game process.

Real-Life Example:

Modern web browsers use multiple threads so that one slow-loading tab doesn't freeze the entire browser.

★ 7. CPU Scheduling Algorithms [IMPORTANT TOPIC]

Explanation:

Beyond basic scheduling, the OS uses specific algorithms to decide CPU time allocation efficiently, balancing fairness, speed, and priority.

Key Points:

- Priority Scheduling -> higher-priority processes run before lower-priority ones.
- Multilevel Queue -> processes are grouped into different queues based on type (like system vs user processes) and scheduled differently.

Example:

An antivirus scan running in the background may be given LOWER priority than the video call app you are actively using, so your call stays smooth.

Real-Life Example:

Task Manager in Windows lets users manually change a program's priority level, directly influencing this exact scheduling decision.

8. Inter-Process Communication (IPC)

Explanation:

IPC refers to the methods that allow different processes to communicate and share data with each other safely, since each process normally has its own isolated memory space.

Key Points:

- Pipes -> a one-way communication channel between processes.
- Shared Memory -> processes share a common block of memory.
- Message Passing -> processes send and receive structured messages.

Example:

A word processor and a spell-check background service might use IPC to send text back and forth for checking, even though they are two separate processes.

Real-Life Example:

Command-line tools in Linux like 'ls | grep' use pipes (the | symbol) as a simple real-world example of IPC.

★ 9. Deadlocks [IMPORTANT TOPIC]

Explanation:

A Deadlock happens when two or more processes are stuck waiting FOREVER for resources held by EACH OTHER, and neither can continue — like two cars blocking each other on a narrow one-lane bridge from opposite directions.

Key Points:

- Mutual Exclusion -> a resource can only be used by one process at a time.
- Hold and Wait -> a process holds one resource while waiting for another.
- No Preemption -> a resource can't be forcibly taken away.
- Circular Wait -> a circular chain of processes, each waiting for the next.

Example:

Process A holds Printer 1 and needs Scanner 1. Process B holds Scanner 1 and needs Printer 1. Neither can proceed — a deadlock.

Real-Life Example:

Operating systems use deadlock detection and prevention techniques to avoid entire systems freezing due to this exact kind of stuck situation.

★ 10. Memory Management *[IMPORTANT TOPIC]*

Explanation:

Memory Management is how the OS keeps track of every part of a computer's RAM — deciding what gets loaded into memory, what gets removed, and making sure different programs don't accidentally overwrite each other's data.

Example:

When you open too many apps at once and your computer slows down, it's often because memory management is working hard to fit everything into limited available RAM.

Real-Life Example:

This is exactly why closing unused apps or browser tabs can speed up a slow computer — it frees up memory for the OS to manage more efficiently.

★ 11. Paging and Segmentation *[IMPORTANT TOPIC]*

Explanation:

Paging and Segmentation are two techniques the OS uses to manage memory by dividing a process into smaller pieces. Paging divides memory into FIXED-size blocks called 'pages'. Segmentation divides a program into VARIABLE-sized, logically meaningful sections (like code, data, stack).

Example:

Paging is like storing a large book by tearing it into equal-sized pages that fit into any available slot on a shelf, regardless of the actual chapter breaks.

Real-Life Example:

Most modern operating systems use paging (or a mix of both) as their primary memory management technique.

★ 12. Virtual Memory *[IMPORTANT TOPIC]*

Explanation:

Virtual Memory lets a computer run programs that need MORE memory than the actual physical RAM available, by using part of the hard disk as 'extra', slower memory when needed.

Example:

If your computer has 8GB of RAM but a program needs 10GB, virtual memory temporarily uses 2GB of hard disk space to make up the difference, letting the program still run (though slower).

Real-Life Example:

This is why running too many heavy programs at once can make your whole computer feel sluggish — it starts relying more on slower virtual memory (disk) instead of fast RAM.

★ 13. File Systems *[IMPORTANT TOPIC]*

Explanation:

A File System is how the OS organizes, names, and stores files and folders on a storage device (like a hard drive), so data can be saved and retrieved reliably.

Key Points:

- NTFS -> commonly used by Windows.
- ext4 -> commonly used by Linux.
- APFS -> used by modern Mac systems.

Example:

When you save a photo to a folder, the file system records exactly WHERE on the physical disk that data is stored, so it can find and load it again later.

Real-Life Example:

Formatting a USB drive lets you choose a file system (like FAT32 or NTFS), which affects which devices can read it.

14. Disk Scheduling

Explanation:

Disk Scheduling algorithms decide the ORDER in which multiple pending requests to read/write data from a disk are handled, to make disk access as fast and efficient as possible.

Key Points:

- FCFS -> handles disk requests in the order they arrive.
- SSTF (Shortest Seek Time First) -> handles the CLOSEST request first, minimizing movement.

Example:

If a hard drive's read head is currently at track 50, and requests come in for tracks 52, 25, and 60, SSTF would serve track 52 first since it requires the least physical movement.

Real-Life Example:

Efficient disk scheduling is a key reason why Solid State Drives (SSDs), and well-optimized traditional hard drives, feel noticeably faster than poorly managed ones.

★ 15. Semaphores and Synchronization [IMPORTANT TOPIC]

Explanation:

When multiple processes or threads access SHARED resources at the same time, problems can occur. A Semaphore is a special variable used to control access to a shared resource, ensuring only a safe number of processes can use it at once — this is called Synchronization.

Example:

A semaphore set to allow only 1 process at a time acts like a single-key bathroom door lock — only one person (process) can be inside using the resource at once.

Real-Life Example:

Synchronization is critical in banking software — without it, two simultaneous withdrawals could incorrectly both succeed even if there wasn't enough balance for both.

16. System Calls

Explanation:

A System Call is how a normal program asks the Operating System to perform a special, protected task on its behalf — like reading a file, creating a new process, or accessing hardware — since programs are not usually allowed to do these things directly.

Example:

When a program wants to read a file, it cannot access the disk directly — it must make a system call, asking the OS to safely perform that file read on its behalf.

Real-Life Example:

Every single file save, network request, or printed document you've ever triggered from an app has relied on system calls happening silently in the background.